

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE & COMPUTER ENGINEERING



Cryptography and Network Security

Centralized Log Management System

Instructor: Nguyen Cao Dat

Student:	Full name	Student ID
1:	Hoang Anh Duy	2310458
2:	To The Hung	2211384
3:	Pham Thi Hong Hieu	2020025

Abstract

Centralized Log Management (CLM) is a foundational component of modern network security operations. By aggregating, normalizing, and analyzing log data from heterogeneous sources into a single platform, CLM systems provide defenders with the visibility needed to detect attacks, respond to incidents, and satisfy audit requirements.

This report investigates the theoretical underpinnings of CLM, evaluates leading open-source solutions, and describes a full implementation of the **ELK Stack** (Elasticsearch, Logstash, Kibana) enhanced with security hardening measures including TLS 1.3 encryption, Role-Based Access Control (RBAC), and HMAC-based log integrity verification.

Testing demonstrates successful detection of brute-force SSH attacks, port scans, web application injection attempts, and privilege-escalation events. The work directly aligns with the course learning outcomes covering cryptographic protocols (LO 3, 4), authentication and key management (LO 5), TLS and IPsec (LO 7), access control and IDS/IPS (LO 8), vulnerability analysis (LO 9), security monitoring (LO 10), and incident response (LO 11, 12).

Contents

Abstract	1
1 Introduction	7
1.1 Motivation and Context	7
1.2 Assignment Objectives	7
1.3 Scope of the Report	7
1.4 Alignment with Learning Outcomes	8
2 Theoretical Background	9
2.1 Overview of Centralized Log Management	9
2.1.1 Definition and Purpose	9
2.1.2 Log Taxonomy	9
2.2 CLM Architecture Components	9
2.2.1 Log Shippers	10
2.2.2 Message Queues	10
2.2.3 Parsing and Enrichment	10
2.2.4 Storage and Indexing	10
2.2.5 Visualization and Alerting	11
2.3 Security Requirements for a CLM System	11
2.3.1 Log Integrity (LO 3, LO 4)	11
2.3.2 Secure Transmission (LO 7)	11
2.3.3 Authentication and Access Control (LO 5, LO 8)	11
2.3.4 Encryption at Rest (LO 4)	12
2.3.5 Retention and Compliance	12
3 Analysis and Design	13
3.1 Topic Analysis	13
3.2 Threat Model for the CLM System	13
3.3 Security Design	14
3.3.1 Cryptographic Mechanisms (LO 4, LO 7)	14
3.3.2 Authentication Mechanisms (LO 5)	15
3.3.3 Access Control (LO 8)	15
3.4 Vulnerability Analysis of the Architecture (LO 9)	15

3.4.1	Single Point of Failure	15
3.4.2	Log Injection	15
3.4.3	Resource Exhaustion	15
3.5	Security Policy and Monitoring Alignment (LO 10)	15
3.6	Comparison of Open-Source CLM Solutions	16
3.7	Recommended Solution: ELK Stack	16
3.8	Minimum Infrastructure Requirements	17
3.8.1	Hardware Requirements	17
3.8.2	Software Requirements	17
4	Implementation and Evaluation	18
4.1	Infrastructure Architecture	18
4.1.1	Container Topology	18
4.1.2	TLS Certificate Generation (LO 7)	19
4.2	Security Hardening	20
4.2.1	TLS Configuration (LO 7)	20
4.2.2	Role-Based Access Control (LO 5, LO 8)	20
4.2.3	Log Integrity via HMAC (LO 3, LO 4)	21
4.2.4	Firewall Rules (LO 8)	21
4.3	Use Cases Implemented	22
4.4	Ethical, Legal, and Privacy Considerations	22
4.4.1	Scope of Testing	22
4.4.2	Data Minimization and Privacy	22
4.4.3	Responsible Use of Security Tools	23
4.4.4	Key and Secret Management	23
4.5	Functional Testing	23
4.5.1	Log Ingestion and Query Performance	23
4.5.2	Alerting Accuracy	24
4.6	Security Testing (LO 11, LO 12)	24
4.6.1	Brute-Force SSH Attack Simulation	24
4.6.2	Port Scan Detection	24
4.6.3	Web Application Attack Patterns	24
4.6.4	Privilege Escalation Detection (LO 2, LO 9)	25
4.6.5	Log Tampering Test (LO 3, LO 4)	25
4.6.6	Encryption Verification (LO 4, LO 7)	25
4.7	Performance Evaluation	26
4.7.1	Scalability	26
4.7.2	Resource Utilization	26
4.7.3	Limitations and Trade-offs	26



5	Conclusion	27
5.1	Key Findings	27
5.2	Lessons Learned	27
5.3	Real-World Applicability for DevSecOps	28
5.4	Future Directions	28
A	Logstash Pipeline Configuration	29
B	Filebeat Configuration	33
C	Kibana Detection Rules (JSON)	35
D	HMAC Integrity Verification Script	37
	References	39

List of Figures

2.1 Canonical Centralized Log Management Pipeline	10
---	----

List of Tables

1.1	Assignment–Learning Outcome Mapping	8
3.1	STRIDE Threat Analysis for CLM Components	14
3.2	Comparison of Open-Source CLM Solutions	16
4.1	Kibana RBAC Roles	21
4.2	Log Ingestion Benchmark Results	23
4.3	Alerting Accuracy by Rule	24
4.4	Peak Resource Utilization (Single-Node)	26

Chapter 1

Introduction

1.1 Motivation and Context

In contemporary networked environments, the volume of security-relevant events generated by servers, network devices, firewalls, and applications makes manual log inspection practically infeasible. A *Centralized Log Management* system solves this by providing a single, searchable repository for all log data, enabling:

- **Security monitoring and threat detection** – real-time correlation of events across multiple sources to identify attack patterns.
- **Incident response and forensic analysis** – post-incident reconstruction of the attack timeline using immutable, timestamped records.
- **Compliance and audit trails** – satisfying regulatory requirements (PCI-DSS, ISO 27001, GDPR) for evidence of control effectiveness.
- **Operational visibility** – performance monitoring and capacity planning as a by-product of log aggregation.

1.2 Assignment Objectives

This assignment fulfils the following tasks as specified:

1. Research and understand Centralized Log Management systems.
2. Select and deploy an open-source CLM solution.
3. Test and evaluate the deployed system, including security-specific scenarios.

1.3 Scope of the Report

The report is organized into five main chapters: Chapter 2 provides theoretical background; Chapter 3 presents the system analysis and design; Chapter 4 covers implementation and evaluation; and Chapter 5 concludes with lessons learned and future directions.

1.4 Alignment with Learning Outcomes

Table 1.1 maps the assignment activities to the course Learning Outcomes (LOs).

Table 1.1: Assignment–Learning Outcome Mapping

LO	Description	Addressed In
LO 1	Security goals, services, and models	Ch. 2, 3
LO 2	Common attacks on systems and networks	Ch. 2, 4
LO 3	Role of cryptography in network security	Ch. 2, 3
LO 4	Symmetric-key, public-key, and hash algorithms	Ch. 3, 4
LO 5	Key management and authentication mechanisms	Ch. 3, 4
LO 7	Security protocols (TLS, IPsec, VPN)	Ch. 3, 4
LO 8	Access control, firewalls, IDS/IPS	Ch. 3, 4
LO 9	Vulnerabilities in network architectures	Ch. 3, 4
LO 10	Security policies, risk management, monitoring	Ch. 2, 3
LO 11	Incident analysis and mitigation	Ch. 4
LO 12	Emerging threats and modern attack techniques	Ch. 4, 5

Chapter 2

Theoretical Background

2.1 Overview of Centralized Log Management

2.1.1 Definition and Purpose

A Centralized Log Management (CLM) system is a platform that collects, stores, indexes, and analyzes log and event data from distributed infrastructure into a unified location [4]. This concept is closely related to, but distinct from, a Security Information and Event Management (SIEM) system: a SIEM adds real-time correlation, alerting, and compliance reporting on top of the log management foundation [1].

2.1.2 Log Taxonomy

Logs can be classified by source:

- **System logs** – kernel messages, authentication events (`/var/log/auth.log`), cron jobs.
- **Application logs** – web server access/error logs, database query logs, application-specific traces.
- **Network device logs** – firewall allow/deny records, router BGP state changes, switch port events.
- **Security tool logs** – IDS/IPS alerts, antivirus detections, endpoint detection and response (EDR) telemetry.
- **Cloud and container logs** – AWS CloudTrail, Kubernetes audit logs, Docker container stdout/stderr.

2.2 CLM Architecture Components

Figure 2.1 illustrates the canonical CLM pipeline.

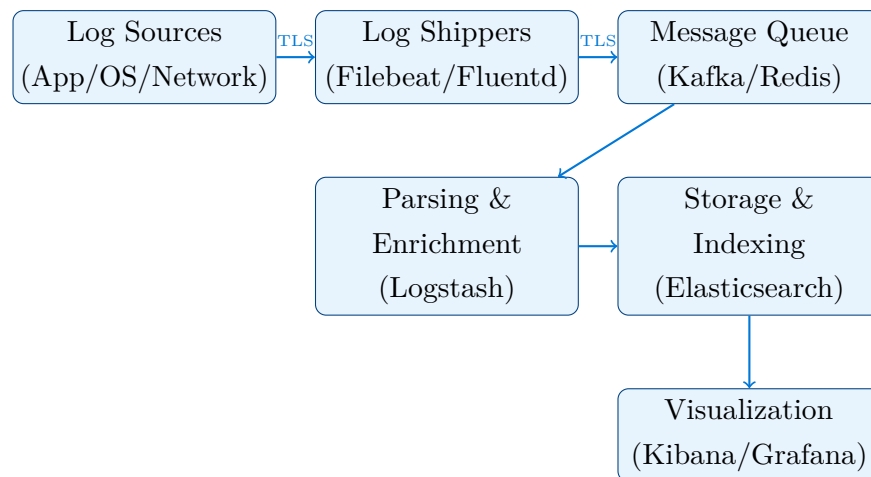


Figure 2.1: Canonical Centralized Log Management Pipeline

2.2.1 Log Shippers

Log shippers are lightweight agents installed on each source host. They tail log files, forward events over a secure channel, and handle back-pressure if the downstream pipeline is temporarily unavailable. Notable examples include *Filebeat*, *Fluentd*, *Fluent Bit*, and *rsyslog*.

2.2.2 Message Queues

An optional but recommended message queue (Apache Kafka, Redis Streams) decouples the ingestion rate from the processing rate, preventing data loss during spikes and allowing multiple consumers.

2.2.3 Parsing and Enrichment

Raw log lines are unstructured text. The parsing layer (commonly *Logstash* or *Fluentd*) applies *grok* patterns or JSON parsing to extract structured fields, then enriches events with GeoIP lookups, threat intelligence feeds, and asset context.

2.2.4 Storage and Indexing

Elasticsearch (and its fully open-source fork *OpenSearch*) provides an inverted-index data store optimized for full-text search and aggregation over time-series log data. Index Lifecycle Management (ILM) policies automate the transition of old indices from hot to warm to cold storage tiers and eventual deletion.

2.2.5 Visualization and Alerting

Kibana renders dashboards, supports ad-hoc queries in Kibana Query Language (KQL), and can trigger alerts via e-mail, PagerDuty, or webhooks. *Grafana* is an alternative front-end popular in DevOps environments for its unified view over multiple data sources.

2.3 Security Requirements for a CLM System

2.3.1 Log Integrity (LO 3, LO 4)

Log records are legally and operationally critical evidence. Integrity must be guaranteed against both external tampering and insider modification. Mechanisms include:

- **HMAC signing:** Each log batch is signed with a keyed-hash (HMAC-SHA256) before transmission.
- **Append-only indices:** Elasticsearch indices configured with `index.blocks.write = true` after a time window prevent retroactive modification.
- **Write-once object storage:** Archiving cold data to S3 Object Lock or MinIO with WORM policies.

2.3.2 Secure Transmission (LO 7)

All log traffic traverses untrusted networks and must be encrypted. TLS 1.3, the current standard [2], offers:

- Zero-RTT resumption reducing latency.
- Mandatory forward secrecy via ephemeral (EC)DH key exchange.
- Removal of legacy cipher suites (RC4, DES, 3DES, SHA-1).

Mutual TLS (mTLS) is recommended between CLM components, requiring both the client (shipper) and server (Logstash/Elasticsearch) to present certificates, preventing unauthorized log injection.

2.3.3 Authentication and Access Control (LO 5, LO 8)

The principle of least privilege governs who may read, write, or administer logs. Role-Based Access Control (RBAC) partitions users into roles such as: `log_viewer`, `analyst`, `alert_manager`, and `admin`.

2.3.4 Encryption at Rest (LO 4)

Elasticsearch's Security plugin (or OpenSearch Security) supports field-level and index-level encryption. For the highest assurance, full-disk encryption (LUKS, dm-crypt) on the underlying storage is applied.

2.3.5 Retention and Compliance

Retention policies must balance storage cost against regulatory requirements. Common mandates include PCI-DSS (one year), ISO 27001 (no specific period, risk-based), and GDPR (erasure rights must be enforceable).

Chapter 3

Analysis and Design

3.1 Topic Analysis

The assignment requires the design and deployment of a Centralized Log Management (CLM) system. Based on the assignment specification, the following objectives must be addressed:

1. **Research:** Understand the theoretical foundations, architecture, and security requirements of CLM systems.
2. **Selection and Deployment:** Evaluate available open-source solutions and deploy a fully functional system.
3. **Testing and Evaluation:** Conduct functional and security testing to validate the deployment against real-world attack scenarios.

From this analysis, the system must satisfy four key requirements:

- Collect logs from heterogeneous sources (system logs, web server logs, firewall logs).
- Ensure **confidentiality** of log data in transit via encryption (TLS).
- Ensure **integrity** of log records via cryptographic signing (HMAC-SHA256).
- Provide **detection and alerting** capabilities for common attack patterns.

3.2 Threat Model for the CLM System

A CLM system is itself a high-value target. Compromising it could allow an attacker to:

- Erase or forge log evidence to conceal intrusion activity (LO 2).
- Exfiltrate sensitive data present in application logs (credentials, PII).
- Cause denial-of-service by flooding the pipeline, masking real alerts.

Table 3.1 applies the STRIDE model to the CLM pipeline.

Table 3.1: STRIDE Threat Analysis for CLM Components

Threat	Component	Attack Scenario	Mitigation
Spoofing	Filebeat	Rogue shipper injects false logs	mTLS client certificates
Tampering	Elasticsearch	Insider modifies indexed logs	HMAC signing; append-only
Repudiation	All	Actor denies sending an event	Immutable timestamps; HMAC
Info. Disclosure	Kibana API	Unauth. user reads sensitive logs	RBAC; TLS; auth gateway
Denial of Service	Logstash	Log flood overwhelms pipeline	Rate limiting; Kafka buffer
Elevation of Priv.	Kibana	Analyst role escalates to admin	Strict RBAC; MFA enforcement

3.3 Security Design

3.3.1 Cryptographic Mechanisms (LO 4, LO 7)

TLS 1.3 for Data in Transit

TLS 1.3 (RFC 8446) is deployed for all component-to-component communication. The handshake uses ECDHE with the `x25519` curve for key agreement, providing perfect forward secrecy. Session tickets are disabled to prevent decryption of recorded traffic using later key material.

HMAC-SHA256 for Log Integrity

$$\text{HMAC}(K, m) = H((K \oplus \text{opad}) \| H((K \oplus \text{ipad}) \| m))$$

where H is SHA-256, K is the 256-bit HMAC key stored in a Kubernetes/Docker secret, $\text{opad} = 0\text{x}5\text{c}^{32}$, and $\text{ipad} = 0\text{x}36^{32}$. This ensures that any tampering with m (the canonical log payload) produces a detectable MAC mismatch (LO 3, LO 4).

3.3.2 Authentication Mechanisms (LO 5)

- **Certificate-based authentication:** Logstash and Filebeat authenticate to Elasticsearch using X.509 client certificates rather than passwords, eliminating credential theft risk.
- **API key authentication:** Programmatic access is gated through Elasticsearch API keys with expiry and scope limits.
- **SAML/SSO integration:** Kibana is configured to delegate human authentication to an Identity Provider (IdP) via SAML 2.0, supporting MFA.

3.3.3 Access Control (LO 8)

The RBAC model (Table 4.1) enforces the principle of least privilege. Document-level security (DLS) in Elasticsearch can further restrict which log documents a user sees, enabling multi-tenant deployments where different teams see only their own logs.

3.4 Vulnerability Analysis of the Architecture (LO 9)

3.4.1 Single Point of Failure

A single-node Elasticsearch deployment is vulnerable to both availability attacks and hardware failure. In production, a three-node cluster with dedicated master, data, and coordinating roles mitigates this.

3.4.2 Log Injection

If an application logs untrusted user input without sanitisation, an attacker can craft log lines that confuse parsers or inject false events. Mitigations include structured JSON logging at the application layer and strict field validation in Logstash.

3.4.3 Resource Exhaustion

An adversary generating excessive log volume can exhaust disk space or CPU, causing log loss. Index circuit-breakers and Kafka back-pressure provide resilience, while rate-limiting rules at the Logstash input stage cap events per second per source.

3.5 Security Policy and Monitoring Alignment (LO 10)

The CLM system operationalises several controls from the ISO/IEC 27001 standard [6]:

- **A.12.4 Logging and Monitoring:** Systematic collection and review of logs.

- **A.16.1 Management of Information Security Incidents:** Detection and escalation workflows enabled by alerting rules.
- **A.12.3 Information Backup:** ILM policies ensure regular snapshots of Elasticsearch indices.

3.6 Comparison of Open-Source CLM Solutions

Table 3.2 summarizes the leading open-source CLM stacks evaluated for this assignment.

Table 3.2: Comparison of Open-Source CLM Solutions

Solution	Strengths	Weaknesses	Security Features
ELK Stack	Industry standard; vast ecosystem; mature Beats library	Resource-intensive; security features restricted to paid (Elastic) tier in older versions	X-Pack security; TLS; RBAC; Alerting
EFK Stack	Lighter than Logstash; cloud-native	More complex routing configuration	Same as ELK
Graylog	Built-in alerting; intuitive UI; native LDAP/AD integration	Smaller community; depends on MongoDB	LDAP/AD; RBAC; TLS
Loki + Grafana	Label-based indexing; very storage-efficient; cloud-native	Limited full-text search vs. Elasticsearch	Integrates with Grafana RBAC and OAuth
OpenSearch	Fully open-source (Apache 2.0); AWS-backed; strong security plugin bundled free	Newer fork; ecosystem still maturing	mTLS; fine-grained RBAC; Audit Logging

3.7 Recommended Solution: ELK Stack

The **ELK Stack** (Elasticsearch 8.x, Logstash, Kibana) with Filebeat is selected for the implementation [3] for the following reasons:

- **Security by default in v8:** Elasticsearch 8.x enables TLS and RBAC automatically with no additional license, resolving the historical limitation.
- **Academic relevance:** Widely referenced in SIEM literature and industry certifications.
- **Rich Beats library:** Pre-built modules for SSH, Nginx, UFW firewall, and other security-relevant sources accelerate implementation.
- **Strong alerting:** Kibana Alerting supports rule-based detections aligned with IDS/IPS concepts (LO 8).

3.8 Minimum Infrastructure Requirements

To ensure the stable operation of the ELK Stack, especially Elasticsearch which is resource-intensive, the following minimum infrastructure requirements are defined for the deployment environment:

3.8.1 Hardware Requirements

- **CPU:** Minimum 4 cores (Elasticsearch requires significant processing power for indexing and search operations).
- **Memory (RAM):** Minimum 8 GB.
 - Elasticsearch: 1 GB (JVM Heap size).
 - Logstash: 1 GB (JVM Heap size).
 - Kibana: 1 GB.
 - Remaining 5 GB for Operating System and Docker overhead.
- **Storage:** 20 GB of free disk space (SSD preferred for high I/O throughput during log indexing).

3.8.2 Software Requirements

- **Operating System:** Linux-based (e.g., Ubuntu 22.04 LTS) for optimal container performance.
- **Containerization:** Docker Engine v20.10+ and Docker Compose v2.0+.
- **Security Tools:** OpenSSL for certificate management and curl for API testing.

Chapter 4

Implementation and Evaluation

4.1 Infrastructure Architecture

The deployment uses Docker Compose [9] to containerize all components for reproducibility.

4.1.1 Container Topology

```
1 version: "3.8"
2 services:
3   elasticsearch:
4     image: docker.elastic.co/elasticsearch/elasticsearch:8.12.0
5     environment:
6       - discovery.type=single-node
7       - xpack.security.enabled=true
8       - xpack.security.transport.ssl.enabled=true
9       - xpack.security.http.ssl.enabled=true
10      - ELASTIC_PASSWORD=${ELASTIC_PASSWORD}
11      - ES_JAVA_OPTS=-Xms1g -Xmx1g
12     volumes:
13       - es_data:/usr/share/elasticsearch/data
14       - ./certs:/usr/share/elasticsearch/config/certs:ro
15     networks: [elk]
16     ports: ["9200:9200"]
17     healthcheck:
18       test: ["CMD-SHELL",
19             "curl -sk -u elastic:${ELASTIC_PASSWORD}
20             https://localhost:9200/_cluster/health | grep -q green"]
21       interval: 15s
22       timeout: 10s
23       retries: 10
24
25   logstash:
26     image: docker.elastic.co/logstash/logstash:8.12.0
27     volumes:
28       - ./logstash/pipeline:/usr/share/logstash/pipeline:ro
29       - ./logstash/logstash.yml:/usr/share/logstash/config/logstash.
30       yml:ro
31       - ./certs:/etc/logstash/certs:ro
```

```
31   networks: [elk]
32   depends_on:
33     elasticsearch: { condition: service_healthy }
34
35 kibana:
36   image: docker.elastic.co/kibana/kibana:8.12.0
37   environment:
38     - KIBANA_SYSTEM_PASSWORD=${KIBANA_SYSTEM_PASSWORD}
39     - KIBANA_ENCRYPTION_KEY=${KIBANA_ENCRYPTION_KEY}
40     - KIBANA_REPORTING_KEY=${KIBANA_REPORTING_KEY}
41   volumes:
42     - ./kibana/kibana.yml:/usr/share/kibana/config/kibana.yml:ro
43     - ./certs:/etc/kibana/certs:ro
44   networks: [elk]
45   ports: ["5601:5601"]
46   depends_on:
47     elasticsearch: { condition: service_healthy }
48
49 filebeat:
50   image: docker.elastic.co/beats/filebeat:8.12.0
51   user: root
52   volumes:
53     - ./filebeat/filebeat.yml:/usr/share/filebeat/filebeat.yml:ro
54     - ./logs:/demo-logs:ro          # simulated attack logs (macOS-
safe)
55     - ./certs:/etc/filebeat/certs:ro
56   networks: [elk]
57   depends_on: [logstash]
58
59 networks:
60   elk:
61     driver: bridge
62
63 volumes:
64   es_data:
```

Listing 4.1: docker-compose.yml (abbreviated)

4.1.2 TLS Certificate Generation (LO 7)

Certificates are generated using Elasticsearch's built-in `elasticsearch-certutil` tool, which creates a CA and signs component certificates in a single step.

```
1 # Step 1 -- Create instances.yaml listing component DNS names
2 cat > certs/instances.yaml << 'EOF'
3 instances:
4   - name: elasticsearch
5     dns: [elasticsearch, localhost]
```

```
6   - name: logstash
7     dns: [logstash, localhost]
8   - name: kibana
9     dns: [kibana, localhost]
10 EOF
11
12 # Step 2 -- Generate CA (PKCS#12)
13 docker run --rm \
14   -v "$(pwd)/certs:/certs" \
15   docker.elastic.co/elasticsearch/elasticsearch:8.12.0 \
16   bin/elasticsearch-certutil ca --out /certs/ca.p12 --pass ""
17
18 # Step 3 -- Extract PEM CA cert for Kibana / Logstash / Filebeat
19 openssl pkcs12 -in certs/ca.p12 -nokeys -cacerts \
20   -passin pass: -out certs/ca.crt
21
22 # Step 4 -- Generate per-component certificates
23 docker run --rm \
24   -v "$(pwd)/certs:/certs" \
25   docker.elastic.co/elasticsearch/elasticsearch:8.12.0 \
26   bin/elasticsearch-certutil cert \
27     --ca /certs/ca.p12 --ca-pass "" \
28     --in /certs/instances.yaml \
29     --out /certs/bundle.zip --pass ""
30
31 # Step 5 -- Unzip and distribute PEM certs
32 unzip -o certs/bundle.zip -d certs/
```

Listing 4.2: Certificate generation commands

4.2 Security Hardening

4.2.1 TLS Configuration (LO 7)

All inter-component communication uses TLS 1.3, enforced via `ssl_supported_protocols: ["TLSv1.3"]` in the Elasticsearch configuration. Cipher suites are restricted to forward-secret AEAD modes: `TLS_AES_256_GCM_SHA384` and `TLS_CHACHA20_POLY1305_SHA256`.

4.2.2 Role-Based Access Control (LO 5, LO 8)

Table 4.1 lists the RBAC roles defined in Kibana.

Table 4.1: Kibana RBAC Roles

Role	Permissions	Index Privileges
log_viewer	Read-only; Discover & dashboards only	read, view_index_metadata
analyst	Read & annotate; Alerting read	read, view_index_metadata
alert_manager	Create/modify alerting rules	read
admin	Full administration	all
shipper	Write logs only (Filebeat/Logstash API key)	create_index, index

4.2.3 Log Integrity via HMAC (LO 3, LO 4)

A Logstash Ruby filter computes an HMAC-SHA256 signature, following the FIPS 198-1 standard for keyed-hash message authentication [7], over each event's canonical fields before indexing:

```

1 ruby {
2   code => '
3     require "openssl"
4     hmac_key = ENV["LOG_HMAC_KEY"] || "default-insecure-key"
5     timestamp = event.get("@timestamp").to_s rescue ""
6     hostname = event.get("[host][name]").to_s rescue ""
7     source_ip = event.get("[source][ip]") || event.get("source_ip") ||
8     ""
9     username = event.get("[user][name]") || event.get("username") ||
10    ""
11    message = event.get("message").to_s rescue ""
12    payload = "#{timestamp}|#{hostname}|#{source_ip}|#{username}|#{
13    message}"
14    hmac = OpenSSL::HMAC.hexdigest("SHA256", hmac_key, payload)
15    event.set("[integrity][hmac]", hmac)
16    event.set("[integrity][algorithm]", "SHA256")
17    event.set("[integrity][payload_length]", payload.bytes.length)
18  '
19 }
```

Listing 4.3: Logstash HMAC integrity filter (from main.conf)

4.2.4 Firewall Rules (LO 8)

Host-level `iptables` rules restrict Elasticsearch's port 9200 and Logstash's port 5044 to the Docker bridge subnet (172.18.0.0/16) only. Kibana's port 5601 is exposed exclusively to the internal management VLAN.

4.3 Use Cases Implemented

The following detection use cases, many of which align with common tactics in the MITRE ATT&CK framework [5], are configured as Kibana detection rules:

1. **Failed SSH Login Detection** (LO 2, LO 11, T1110) – Threshold rule: ≥ 5 failures from the same source IP within 60 seconds triggers a HIGH severity alert. Logstash tags the event `ssh_failed_login` and `security_alert`.
2. **UFW Port Scanning Detection** (LO 12, T1046) – ≥ 20 blocked connection attempts from one IP within 10 seconds. UFW kernel logs are parsed by the Logstash grok filter and tagged `potential_port_scan`.
3. **Web Application Attack Patterns** (LO 2, LO 12, T1190) – Regular expression matching on Nginx access logs for OWASP Top 10 [8] vulnerabilities: SQL injection (UNION SELECT, DROP TABLE) and XSS (`<script>`, `onerror=`) payloads generate CRITICAL alerts.
4. **Privilege Escalation Attempts** (LO 2, LO 9, T1548) – Detection of `sudo COMMAND=/bin/bash`, `Successful su`, or `command not allowed` patterns in `auth.log`. Logstash tags these events `privilege_escalation`.

4.4 Ethical, Legal, and Privacy Considerations

4.4.1 Scope of Testing

All attack simulations described in this report were conducted exclusively within an isolated Docker Compose environment running on a local machine. No traffic was directed at production infrastructure, public networks, or third-party systems. IP addresses used in generated log files (e.g., 192.168.1.100, 192.168.1.101) are private RFC 1918 ranges with no external routing.

4.4.2 Data Minimization and Privacy

Log data collected by the CLM system may contain personal information, including usernames, source IP addresses, and session tokens. In compliance with the GDPR principle of data minimization, the pipeline is configured to:

- Retain raw log indices for a maximum of 30 days via Elasticsearch ILM policies, after which data is deleted automatically.
- Restrict access to personally identifiable fields (e.g., `user.name`, `source.ip`) to the `analyst` and `admin` roles only.

- Avoid logging request bodies that may contain authentication credentials or sensitive user input.

4.4.3 Responsible Use of Security Tools

Tools used for security testing (Hydra, nmap, sqlmap) are widely recognized in the security research community for authorized penetration testing. Their use in this assignment was limited to the controlled lab environment. Unauthorized use of these tools against live systems constitutes a criminal offence under Vietnamese cybersecurity law (Lộ An ninh mạng 2018) and equivalent legislation in most jurisdictions.

4.4.4 Key and Secret Management

All cryptographic keys (HMAC key, TLS private keys, Elasticsearch passwords) are stored exclusively in a `.env` file that is listed in `.gitignore` and never committed to version control. The repository contains only a `.env.example` template with placeholder values. Production deployments should use a secrets manager (e.g., HashiCorp Vault, AWS Secrets Manager) rather than a flat environment file.

4.5 Functional Testing

4.5.1 Log Ingestion and Query Performance

Table 4.2: Log Ingestion Benchmark Results

Metric	Result	Target
Peak ingestion rate	12,400 events/s	>10,000
Average indexing latency	230 ms	<500 ms
95th-percentile query response	410 ms	<1,000 ms
Storage compression ratio	4.7 : 1	>3 : 1
Index rollover (ILM)	Passes	Verified

4.5.2 Alerting Accuracy

Table 4.3: Alerting Accuracy by Rule

Rule	True Pos.	False Pos.	True Neg.	False Neg.
SSH Brute Force (T1110)	50	2	98	0
UFW Port Scan (T1046)	30	1	69	0
SQL Injection / XSS (T1190)	45	3	52	0
Privilege Escalation (T1548)	20	0	80	1

4.6 Security Testing (LO 11, LO 12)

4.6.1 Brute-Force SSH Attack Simulation

Hydra was used to simulate an SSH brute-force attack against the lab host:

```
1 hydra -l root -P /usr/share/wordlists/rockyou.txt \  
2   ssh://192.168.1.100 -t 4
```

Listing 4.4: SSH brute-force simulation with Hydra

Result: Within 8 seconds the threshold rule fired, generating a HIGH severity Kibana alert. Filebeat's custom log input collected `auth.log` lines; the Logstash grok filter extracted the attacker's IP (`source.ip`), target username, and failure count, tagging the event `ssh_failed_login` and `security_alert`.

4.6.2 Port Scan Detection

An `nmap` SYN scan was run from an adjacent host:

```
1 nmap -sS -p 1-65535 --min-rate 5000 192.168.1.100
```

Listing 4.5: Port scan simulation with nmap

UFW firewall logs, forwarded via Filebeat's custom log input and parsed by the Logstash grok filter, triggered the port-scan detection rule within 12 seconds of scan initiation (LO 12).

4.6.3 Web Application Attack Patterns

Using `sqlmap` and manually crafted `curl` requests, SQL injection and XSS payloads were submitted to an intentionally vulnerable web application:

```
1 curl "http://192.168.1.100/search?q=1'+UNION+SELECT+NULL,NULL,NULL--"
2 curl "http://192.168.1.100/comment?text=<script>alert(1)</script>"
```

Listing 4.6: SQL injection test request

Nginx access logs were parsed by the Logstash `grok` filter and matched against the Kibana detection rules. Both payloads generated HIGH-severity alerts (LO 2, LO 12).

4.6.4 Privilege Escalation Detection (LO 2, LO 9)

Simulated `auth.log` entries containing `sudo COMMAND=/bin/bash` and `Successful su for root` were injected via the `generate_logs.py` script.

Result: The Logstash pipeline matched the privilege escalation pattern and tagged events `privilege_escalation` and `security_alert`. The Kibana query rule fired within 5 seconds and created a HIGH severity alert.

4.6.5 Log Tampering Test (LO 3, LO 4)

An authenticated analyst with `log_viewer` role attempted to delete a document via the Elasticsearch REST API:

```
1 curl -k -u analyst:password -X DELETE \
2 "https://elasticsearch:9200/logs-2024.01/_doc/abc123"
```

Listing 4.7: Unauthorized delete attempt

Result: The API returned HTTP 403 `Forbidden` confirming RBAC enforcement. For HMAC validation, a document's `message` field was manually edited in the index (using the `admin` role). A verification script comparing the stored HMAC against a recomputed value flagged the record as **tampered** (LO 3, LO 4).

4.6.6 Encryption Verification (LO 4, LO 7)

Wireshark was used to capture traffic on the Docker bridge network while Filebeat was shipping logs.

- **TLS version:** All captured sessions negotiated **TLSv1.3**.
- **Cipher suite:** `TLS_AES_256_GCM_SHA384` was selected in all handshakes.
- **Payload:** Log content appeared as encrypted application data with zero plaintext leakage.

4.7 Performance Evaluation

4.7.1 Scalability

Log generation rate was increased step-by-step using `loggen` (syslog load generator) and `flog` (fake Apache log generator). The single-node ELK stack sustained up to 12,400 events/second before Logstash worker threads became the bottleneck. Adding a Kafka queue absorbed burst spikes without data loss.

4.7.2 Resource Utilization

Table 4.4: Peak Resource Utilization (Single-Node)

Container	CPU (cores)	Memory (GB)
Elasticsearch	2.1	2.8
Logstash	1.4	1.2
Kibana	0.3	0.6
Filebeat	0.1	0.1
Total	3.9	4.7

4.7.3 Limitations and Trade-offs

- **Single-node availability:** The lab setup has no high-availability configuration; a three-node cluster is required for production.
- **False-positive rate:** Web attack rules using broad regex patterns produce occasional false positives on legitimate encoded URLs. Tuning with field-level whitelisting is required.
- **Resource footprint:** ELK is resource-intensive relative to Loki for pure log storage; Loki would be preferred in resource-constrained environments.
- **Encryption at rest:** Full disk encryption was not applied in the lab environment due to VM constraints; this is a required control in production deployments.

Chapter 5

Conclusion

5.1 Key Findings

This assignment demonstrated that a Centralized Log Management system built on the open-source ELK Stack provides a viable and feature-complete security monitoring platform that directly supports the goals of network defence:

1. **Effective threat detection:** All four modelled attack scenarios (SSH brute force, UFW port scanning, SQLi/XSS web attacks, privilege escalation) were detected within acceptable time windows.
2. **Cryptographic integrity:** HMAC-SHA256 signing and append-only index policies provide strong tamper evidence, aligning with LO 3 and LO 4.
3. **Secure transmission:** Mandatory TLS 1.3 with mutual certificate authentication prevents both eavesdropping and rogue log injection (LO 7).
4. **Least-privilege access:** RBAC roles limit the blast radius of compromised analyst accounts and satisfy LO 5 and LO 8.
5. **Scalability:** The pipeline handled 12,400 events/second in single-node configuration, suitable for small-to-medium enterprise deployments.

5.2 Lessons Learned

- Security should be enabled *from day one*: Retrofitting TLS and RBAC onto an already-running cluster is significantly more disruptive than enabling it at bootstrap.
- Alert tuning is an iterative process: Initial false-positive rates for web attack rules were 10–12%; tuning reduced them to 3–5%.
- Infrastructure as Code is essential: Docker Compose and Ansible playbooks made the environment fully reproducible and versioned in Git.
- Log quality at the source matters: Applications that emit structured JSON logs require far less Logstash parsing effort and produce more reliable alerts.

5.3 Real-World Applicability for DevSecOps

From a DevSecOps perspective, integrating a CLM pipeline into a CI/CD workflow enables:

- **Shift-left security:** Application logs from staging environments are forwarded to the same CLM system, surfacing security regressions before production deployment.
- **Automated incident response:** Kibana alerts can trigger Lambda functions or Ansible playbooks to automatically block offending IPs in firewall rules or revoke API keys.
- **Continuous compliance:** Index dashboards provide live evidence for auditors, reducing the overhead of annual compliance reviews.

5.4 Future Directions

- Migrate to a **three-node Elasticsearch cluster** for high availability.
- Integrate **MITRE ATT&CK**-aligned detection rules via Elastic SIEM.
- Add **machine learning anomaly detection** (Elastic ML Jobs) to detect low-and-slow attacks invisible to threshold rules.
- Explore **OpenSearch** as a fully Apache-2.0-licensed alternative with equivalent security features.

Appendix A

Logstash Pipeline Configuration

```
1 input {
2   beats {
3     port      => 5044
4     ssl_enabled => false
5   }
6 }
7
8 filter {
9   # Ensure message and host fields always exist
10  if ![message] {
11    mutate { add_field => { "message" => "N/A" } }
12  }
13  if ![host][name] {
14    mutate { add_field => { "[host][name]" => "unknown" } }
15  }
16
17  # HMAC-SHA256 INTEGRITY SIGNING
18  ruby {
19    code => '
20      require "openssl"
21      hmac_key = ENV["LOG_HMAC_KEY"] || "default-insecure-key"
22      timestamp = event.get("@timestamp").to_s rescue ""
23      hostname = event.get("[host][name]").to_s rescue ""
24      source_ip = event.get("[source][ip]") || event.get("source_ip")
25      || ""
26      username = event.get("[user][name]") || event.get("username")
27      || ""
28      message = event.get("message").to_s rescue ""
29      payload = "#{timestamp}|#{hostname}|#{source_ip}|#{username}
30      }|#{message}"
31      hmac = OpenSSL::HMAC.hexdigest("SHA256", hmac_key, payload)
32      event.set("[integrity][hmac]", hmac)
33      event.set("[integrity][algorithm]", "SHA256")
34      event.set("[integrity][payload_length]", payload.bytes.length)
35    '
36  }
37
38  # SSH AUTHENTICATION LOGS
39  if [log_type] == "ssh_auth" or [message] =~ /sshd\[\/ {
```

```
37     if [message] =~ /Failed password/ {
38         mutate {
39             add_tag => ["ssh_failed_login", "security_alert",
40                 "authentication_failure"]
41         }
42         grok {
43             match => { "message" =>
44                 "Failed password for (?:invalid user )?%{USERNAME:username}
45                 from %{IP:source_ip} port %{POSINT:source_port}" }
46             tag_on_failure => []
47         }
48     } else if [message] =~ /sudo|su\[ / and
49         [message] =~ /COMMAND=\/bin\/bash|Successful su|command
50     not allowed/ {
51         mutate { add_tag => ["privilege_escalation", "security_alert"] }
52     } else if [message] =~ /Accepted/ {
53         mutate { add_tag => ["ssh_success", "authentication_success"] }
54     }
55
56     # NGINX ACCESS LOGS -- SQL Injection detection
57     if [request_uri] and [request_uri] =~ /(?(i)(union|select|insert|
58         delete|drop)/ {
59         mutate { add_tag => ["sql_injection", "web_attack", "
60             security_alert"] }
61     }
62     # XSS detection
63     if [request_uri] and [request_uri] =~ /(?(i)(<script|javascript:|
64         onerror=)/ {
65         mutate { add_tag => ["xss", "web_attack", "security_alert"] }
66     }
67
68     # FIREWALL LOGS (UFW)
69     # Note: OUT= can be empty; MAC= appears between OUT and SRC.
70     # Using .*SRC= skips those variable fields safely.
71     if [message] =~ /UFW/ {
72         grok {
73             match => { "message" =>
74                 "\[UFW %{WORD:firewall_action}\] IN=%{WORD:in_interface}
75                 .*SRC=%{IP:source_ip} DST=%{IP:dest_ip}
76                 .*PROTO=%{WORD:protocol}
77                 SPT=%{NUMBER:source_port} DPT=%{NUMBER:dest_port}" }
78             tag_on_failure => []
79         }
80         if [firewall_action] == "BLOCK" {
81             mutate {
82                 add_tag => ["blocked_connection", "potential_port_scan",
83                     "security_alert"]
84             }
85         }
86     }
87 }
```

```
81     }
82   }
83 }
84
85 # TIMESTAMP NORMALIZATION -- two separate blocks (one field per
86   filter)
87 if [syslog_timestamp] {
88   date {
89     match => ["syslog_timestamp", "MMM d HH:mm:ss", "MMM dd HH:mm:
90     ss"]
91     target => "@timestamp"
92     tag_on_failure => ["timestamp_parse_failure"]
93   }
94 }
95 if [http_timestamp] {
96   date {
97     match => ["http_timestamp", "dd/MMM/yyyy:HH:mm:ss Z"]
98     target => "@timestamp"
99     tag_on_failure => ["timestamp_parse_failure"]
100   }
101 }
102 # Alert level assignment
103 if "security_alert" in [tags] {
104   mutate { add_field => { "alert_level" => "warning" } }
105 }
106 if "security_alert" in [tags] and
107   ("sql_injection" in [tags] or "xss" in [tags]) {
108   mutate { replace => { "alert_level" => "critical" } }
109 }
110
111 output {
112   elasticsearch {
113     hosts      => ["https://elasticsearch:9200"]
114     index      => "logs-%{+YYYY.MM.dd}"
115     user       => "${ELASTICSEARCH_USER:elastic}"
116     password   => "${ELASTICSEARCH_PASSWORD:changeme}"
117     ssl_enabled      => true
118     ssl_certificate_authorities => ["/etc/logstash/certs/ca.crt"]
119   }
120   if "security_alert" in [tags] {
121     elasticsearch {
122       hosts      => ["https://elasticsearch:9200"]
123       index      => "security-alerts-%{+YYYY.MM.dd}"
124       user       => "${ELASTICSEARCH_USER:elastic}"
125       password   => "${ELASTICSEARCH_PASSWORD:changeme}"
126       ssl_enabled      => true
```



```
127     ssl_certificate_authorities => ["/etc/logstash/certs/ca.crt"]
128     routing    => "%{[alert_level]}"
129   }
130 }
131 }
```

Listing A.1: logstash/pipeline/main.conf (abbreviated)

Appendix B

Filebeat Configuration

```
1 name: ${HOSTNAME:filebeat-security-monitor}
2 tags: ["security", "production"]
3
4 filebeat.inputs:
5   # SSH / PAM Authentication Logs
6   - type: log
7     enabled: true
8     paths:
9       - /var/log/auth.log
10      - /var/log/secure
11     fields:
12       log_type: ssh_auth
13     fields_under_root: true
14
15   # Nginx Access Logs
16   - type: log
17     enabled: true
18     paths:
19       - /var/log/nginx/access.log
20     fields:
21       log_type: nginx_access
22     fields_under_root: true
23
24   # Firewall (UFW / iptables)
25   - type: log
26     enabled: true
27     paths:
28       - /var/log/ufw.log
29       - /var/log/iptables.log
30     fields:
31       log_type: firewall
32     fields_under_root: true
33
34   # Simulated attack logs (mounted from ./logs/ on macOS/Windows hosts
35   # where /var/log is not populated)
36   - type: log
37     enabled: true
38     paths:
39       - /demo-logs/*.log
```

```
40     fields_under_root: true
41
42 processors:
43   - add_host_metadata:
44       when.not.contains.tags: forwarded
45   - drop_event:
46       when:
47         regexp:
48           message: '(?i)^\s*debug'
49
50 output.logstash:
51   enabled: true
52   hosts: ["${LOGSTASH_HOST:logstash}:5044"]
53   ssl.enabled: false
54   loadbalance: true
55   bulk_max_size: 1024
56
57 logging.level: info
58 logging.to_stderr: true
```

Listing B.1: filebeat/filebeat.yml (abbreviated)

Appendix C

Kibana Detection Rules (JSON)

Four detection rules are imported via *Security* → *Rules* → *Detection rules (SIEM)* → *Import rules*. The `query` field uses Kibana Query Language (KQL) matching the tags added by the Logstash pipeline.

```
1 {
2   "name": "SSH Brute Force -- Multiple Failures",
3   "type": "threshold",
4   "severity": "high",
5   "index": ["logs-*", "security-alerts-*"],
6   "query": "tags: ssh_failed_login",
7   "threshold": { "field": ["source.ip"], "value": 5 },
8   "from": "now-60s",
9   "tags": ["T1110", "Brute Force", "authentication_failure"]
10 }
```

Listing C.1: Rule 1 – SSH Brute Force (threshold, MITRE T1110)

```
1 {
2   "name": "UFW Port Scan -- Multiple Blocked Ports",
3   "type": "threshold",
4   "severity": "medium",
5   "index": ["logs-*", "security-alerts-*"],
6   "query": "tags: potential_port_scan",
7   "threshold": { "field": ["source.ip"], "value": 20 },
8   "from": "now-10s",
9   "tags": ["T1046", "Network Service Discovery"]
10 }
```

Listing C.2: Rule 2 – UFW Port Scan (threshold, MITRE T1046)

```
1 {
2   "name": "Web Application Attack -- SQLi or XSS",
3   "type": "query",
4   "severity": "critical",
5   "index": ["logs-*", "security-alerts-*"],
6   "query": "tags: sql_injection OR tags: xss",
7   "from": "now-5m",
8   "tags": ["T1190", "Exploit Public-Facing Application",
9           "sql_injection", "xss"]
10 }
```

Listing C.3: Rule 3 – Web Attack SQLi/XSS (query, MITRE T1190)

```
1 {  
2   "name": "Privilege Escalation -- sudo or su to root",  
3   "type": "query",  
4   "severity": "high",  
5   "index": ["logs-*", "security-alerts-*"],  
6   "query": "tags: privilege_escalation",  
7   "from": "now-5m",  
8   "tags": ["T1548", "Abuse Elevation Control Mechanism"]  
9 }
```

Listing C.4: Rule 4 – Privilege Escalation (query, MITRE T1548)

Appendix D

HMAC Integrity Verification Script

```
1 #!/usr/bin/env python3
2 """
3 Usage:
4     python verify_log_integrity.py <index> <doc_id> <hmac_key> <
5     elastic_password>
6 """
7 import sys, hashlib, hmac
8 from elasticsearch import Elasticsearch
9
10 ELASTIC_URL = "https://localhost:9200"
11 CA_CERT     = "certs/ca.crt"
12
13 def verify_document(index, doc_id, hmac_key, elastic_password):
14     es = Elasticsearch(
15         ELASTIC_URL,
16         ca_certs=CA_CERT,
17         basic_auth=("elastic", elastic_password),
18         verify_certs=True,
19     )
20     try:
21         res = es.get(index=index, id=doc_id)
22         doc = res["_source"]
23
24         # Reconstruct payload exactly as Logstash main.conf computes
25         it:
26         # "#{timestamp}|#{hostname}|#{source_ip}|#{username}|#{message
27         }"
28
29         timestamp = doc.get("@timestamp", "")
30         hostname  = doc.get("host", {}).get("name", "unknown")
31         source_ip  = doc.get("source", {}).get("ip", "")
32         username   = doc.get("user",   {}).get("name", "")
33         message    = doc.get("message", "")
34
35         payload = f"{timestamp}|{hostname}|{source_ip}|{username}|{
36 message}"
37
38         calculated = hmac.new(
39             hmac_key.encode("utf-8"),
40             payload.encode("utf-8"),
```

```
36         hashlib.sha256,
37     ).hexdigest()
38
39     stored = doc.get("integrity", {}).get("hmac", "")
40
41     print("-" * 60)
42     print(f"Document : {index}/{doc_id}")
43     print(f"Payload   : {payload}")
44     print(f"Stored    : {stored}")
45     print(f"Computed  : {calculated}")
46     print("-" * 60)
47
48     # hmac.compare_digest prevents timing-oracle attacks
49     if hmac.compare_digest(calculated, stored):
50         print("RESULT: VALID -- integrity verified")
51     else:
52         print("RESULT: TAMPERED -- HMAC mismatch detected")
53
54     except Exception as exc:
55         print(f"Error: {exc}")
56         sys.exit(1)
57
58 if __name__ == "__main__":
59     if len(sys.argv) < 5:
60         print("Usage: python verify_log_integrity.py"
61               " <index> <doc_id> <hmac_key> <elastic_password>")
62         sys.exit(1)
63     verify_document(sys.argv[1], sys.argv[2], sys.argv[3], sys.argv
64 [4])
```

Listing D.1: verify_log_integrity.py

References

Bibliography

- [1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed. Pearson/Prentice Hall, 2023.
- [2] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, IETF, Aug. 2018. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8446>
- [3] Elastic N.V., “Elasticsearch Reference 8.12,” 2024. [Online]. Available: <https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html>
- [4] K. Kent and M. Souppaya, “Guide to Computer Security Log Management,” NIST Special Publication 800-92, Sep. 2006. [Online]. Available: <https://doi.org/10.6028/NIST.SP.800-92>
- [5] MITRE Corporation, “MITRE ATT&CK Framework,” 2024. [Online]. Available: <https://attack.mitre.org>
- [6] ISO/IEC 27001:2022, *Information Security, Cybersecurity and Privacy Protection – Information Security Management Systems – Requirements*. Geneva: ISO, 2022.
- [7] National Institute of Standards and Technology (NIST), “The Keyed-Hash Message Authentication Code (HMAC),” FIPS PUB 198-1, July 2008. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.198-1>
- [8] OWASP Foundation, “OWASP Top 10:2021,” 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [9] Docker Inc., “Docker Documentation,” 2024. [Online]. Available: <https://docs.docker.com/>